

1. How are the addresses decided for the following variables a and x, i.e., during the runtime, how does the program know the address of these two variables?

```
void foo(int a) {  
    int x;}  
  
Address of a = ebp + 8  
Address of x = esp - 4
```

2. In which memory segments are the variables in the following code located?

```
int i = 0;  
  
void func(char *str) {  
    char *ptr = malloc(sizeof(int));char buf[1024];  
    int j;  
    static int y;  
}
```

i is an initialized global variable, in data segment

str, ptr, buf and j are on the stack

The data pointed to by ptr is on the heap.

y is a static uninitialized variables, in BSS.

3. Please draw the function stack frame for the following C function.

```
int bof(char *str) {  
    char buffer[24];strcpy(buffer,str);return 1;  
}
```

str
Return address
Previous ebp
Buffer[23]
...

Buffer[0]

4. A student proposes to change how the stack grows. Instead of growing from high address to low address, the student proposes to let the stack grow from low address to high address. This way, the buffer will be allocated above the return address, so overflowing the buffer will not be able to affect the return address. Please comment on this proposal.

This wouldn't defeat the problem, only change the nature of the attack. While you wouldn't be able to hijack the current stack frame, you still override the return address of the next stack frame. For example, say you have functions:

```
void bar(char* str) {  
    char c[7];  
    strcpy(c, str);  
}  
void foo() {  
    bar("overflow");  
}
```

- Normally, in the case where the stack grows from high to low, if you passed a string too large into the bar function, it could override the return address of the bar function.

- But if the stack grew from low to high, you wouldn't override the bar function but could instead override the return address of the strcpy function.

5. In the buffer overflow example shown in Listing 4.1, the buffer overflow occurs inside the strcpy() function, so the jumping to the malicious code occurs when strcpy() returns, not when foo() returns. Is this true or false? Please explain.

The answer is false. Yes, the buffer overflow occurs in strcpy, but what the buffer overflow does is override the return address of the foo function, so the jumping to the malicious code occurs when foo() returns, not when strcpy() returns.

6. The buffer overflow example was fixed as below. Is this safe ?

```
int bof(char *str, int size)  
{  
    char buffer = (char*) malloc(size);  
    strcpy(buffer, str);  
    return 1;  
}
```

Yes, buffer is now allocated on the heap, and you can't reach the stack by overflowing something on the heap.

7. Several students had issue with the buffer overflow attack. Their bad file was constructed properly where shell code is at the end of bad file, but when they try different return addresses, they get the following observations. Can you explain why some addresses work and some do not?

buffer address : 0xbffff180

```
case 1 : long retAddr = 0xbffff250 -> Able to get shell
accesscase 2 : long retAddr = 0xbffff280 -> Able to get shell
accesscase 3 : long retAddr = 0xbffff300 -> Cannot get shell
accesscase 4 : long retAddr = 0xbffff310 -> Able to get shell
accesscase 5: long retAddr = 0xbffff400 -> Cannot get shell
access
```

8. The following function is called in a privileged program. The argument str points to a string that is entirely provided by users (the size of the string is up to 300 bytes). When this function is invoked, the address of the buffer array is 0xAABB0010, while the return address is stored in 0xAABB0050. Please write down the string that you would feed into the program, so when this string is copied to buffer and when the `bof()` function returns, the privileged program will run your code. In your answer, you don't need to write down the injected code, but the offsets of the key elements in your string need to be correct. Note: there is a trap in this problem; some people may be lucky and step over it, but some people may fall into it. Be careful.

```
int bof(char *str) {
    char buffer[24];strcpy(buffer,str);return 1;
}
```

RT ... RT NOP ... NOP (Malicious code)

Buffer start at 0xAABBCC10

The return address is at 0xAABBCC10+X+8

X is in the range of 20...100

Spray the first 108+4 bytes of the return address, i.e 28 return addresses

The malicious code is of 24 bytes, located at the end of 300 byte buffer

The return address can be any address in the NOP range, that does not contain a zero byte. A safe choice is $0xAABBCC10+100+8+4 = 0xAABBCC80$

9. Why does ASLR make buffer-overflow attack more difficult?

ASLR randomizes the start location of the stack, i.e, every time the code is loaded in memory the stack address changes. This makes it harder to guess stack address in memory, and accordingly makes it hard to guess the `ebp` address as well as the address of the malicious code.

10. To write a shellcode, we need to know the address of the string `"/bin/sh"`. If we have to hardcode the address in the code, it will become difficult if ASLR is turned on. Shellcode solved that problem without hardcoding the address of the string in the code. Please explain how the shellcode in `exploit.c` (Listing 4.2) achieved that.

```
shellcode= (  
    "\x31\xc0"      # xorl    %eax,%eax  
    "\x50"          # pushl   %eax  
    "\x68" "//sh"    # pushl   $0x68732f2f  
    "\x68" "/bin"    # pushl   $0x6e69622f  
    "\x89\xe3"      # movl    %esp,%ebx  
    "\x50"          # pushl   %eax  
    "\x53"          # pushl   %ebx  
    "\x89\xe1"      # movl    %esp,%ecx  
    "\x99"          # cdq  
    "\xb0\x0b"      # movb    $0x0b,%al  
    "\xcd\x80"      # int     $0x80  
).encode(' latin-1')
```

- This approach: This is to not use any hard-coded address in our program.
- We push this `"/sh"` and `"/bin"` onto the stack, and calculate it during the running process with a zero byte marking the end of the string. At that time, stack pointer `esp` -> the string address

11. When you construct the attack string in Problem 8, you have many choices when deciding what value to put in the return address field. What is the smallest value that you can use?